Dokumentation

ELMAR
CNC-Cutter

Autoren:	Alexandra Roth 7025637 David Atrops 7013979 Dennis Smirnow 7023434 Leon Ronge 7026641 Samuel Drees 7026354
Studiengang:	Maschinenbau und Design
Erstprüfer:	Prof. Dr. Elmar Wings Malena Wilken
Abgabedatum:	28. April 2026

Hochschule Emden/Leer · Fachbereich Technik · Abteilung Maschinenbau
Constantiaplatz 4 · 26723 Emden · http://www.hs-emden-leer.de

Inhaltsverzeichnis

Inhaltsverzeichnis 3

Abbildungsverzeichnis 5

I. Domain Knowledge - Application 7

II. Domain Knowledge - Hardware 9

1. Sensor: Endschalter 11
1.1. Einleitung: Positionssensoren 11
1.2. Definition und Funktion: Elektromechanischer Endschalter 11
1.3. Specification 11
1.4. Verwendung und Schaltfunktionen 12
1.5. Analyse: Vor- und Nachteile 13
1.6. Ausblick: Intelligente und vernetzte Sensorik 14
1.7. Library 14
1.8. Beispiel 14
1.8.1. Erklärung des Beispiel-Codes 14
1.9. Einfacher Test-Code 15
1.10. Einfache Anwendung 16

III. Domain Knowledge - Tools 19

IV. Developmenmt 21

V. Monitoring 23

VI. Verification - Evaluation - Conclusion 25

VII.Anhang 27

Literaturverzeichnis 29

Literaturverzeichnis 29

Abbildungsverzeichnis

1.1. Darstellung des im Projekt verwendeten Endschalters 12
1.2. Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO 13

Teil I.

Domain Knowledge - Application

Teil II.

Domain Knowledge - Hardware

1. Sensor: Endschalter

1.1. Einleitung: Positionssensoren

In der industriellen Fertigungssteuerung nimmt die Erfassung von Zuständen eine zentrale Rolle ein. Gemäß der Fachliteratur wandelt ein Sensor (lat. sensus, der Sinn) „eine physikalische Größe (z. B. Kraft oder Temperatur) mit Hilfe eines physikalischen Effekts in ein elektrisches Signal um“ [Her+, S. 433]. Speziell im Maschinenbau wird bei der Steuerung zwischen zwei grundlegenden Messverfahren unterschieden: Der „Positionserfassung durch Schalter“ und der „Positionserfassung durch Wegbeobachtung“. [Her+, S. 435]
Bevor der Fokus auf kontaktbehaftete Schalter gelegt wird, sind folgende berührungslose Sensoren (Sensorschalter) als moderner Industriestandard zu nennen:

- Induktive Sensoren: Erfassen metallische Elemente „ohne Kontakt zwischen der Bewegung und dem Sensor“ [Her+, S. 436].
- Kapazitive Sensoren: Nutzen die Beeinflussung eines elektrischen Feldes zur Kapazitätsänderung [Her+, S. 438].
- Optische Sensoren: Arbeiten mit Infrarot-Lichtstrahlen zur Objekterkennung (z. B. Lichttaster oder Lichtschranken) [Her+, S. 440].
- Ultraschall-Sensoren: Basieren auf der Messung der Laufzeit von Schallimpulsen [Her+, S. 443].

1.2. Definition und Funktion: Elektromechanischer Endschalter

Ein elektromechanischer Endschalter ist ein kontaktbehafteter Positionssensor. Ein solcher Schalter „hat die Aufgabe, das Erreichen einer bestimmten Position (Endlage) zu melden oder einen Schaltvorgang auszulösen“ [Her+, S. 435]. Dabei wird durch das Betätigen eines mechanischen Elements, wie beispielsweise eines Hebels oder Tasters, ein elektrischer Kontakt geöffnet oder geschlossen [Her+, S. 120–122].
Technische Spezifikationen (Beispiel Creality): Bei dem vorliegenden Bauteil handelt es sich um einen „originalen Endschalter von Creality 3D, der bei fast allen FDM Druckern von Creality 3D verbaut ist“ [3dj].

1.3. Specification

- Kompatibilität: Geeignet für die X-, Y- oder Z-Achse von Modellen wie der Ender-3 oder CR-10 Serie [3dj][Bas].
- Hersteller: Creality
- Hersteller SKU: 4004180004
- Gewicht: 7.3g
- Abmaße

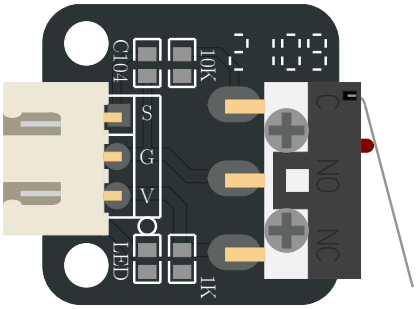

Abbildung 1.1.: Darstellung des im Projekt verwendeten Endschalters

1.4. Verwendung und Schaltfunktionen

Mechanische Endschalter werden zur Überwachung von Bewegungsabläufen eingesetzt. Dabei können verschiedene logische Funktionen realisiert werden:

- Schließer: Der Kontakt wird bei Betätigung geschlossen.
- Öffner: Der Kontakt wird bei Betätigung unterbrochen.
- Antivalenter Sensor: Dieser verfügt über zwei Ausgänge, wobei „stets ein Kontakt geschlossen und der andere offen“ ist [Her+, S. 435]. Dies ermöglicht neben der reinen Schaltfunktion auch eine „Fehlererkennung und Diagnose durch die SPS“ [Her+, S. 435–436].

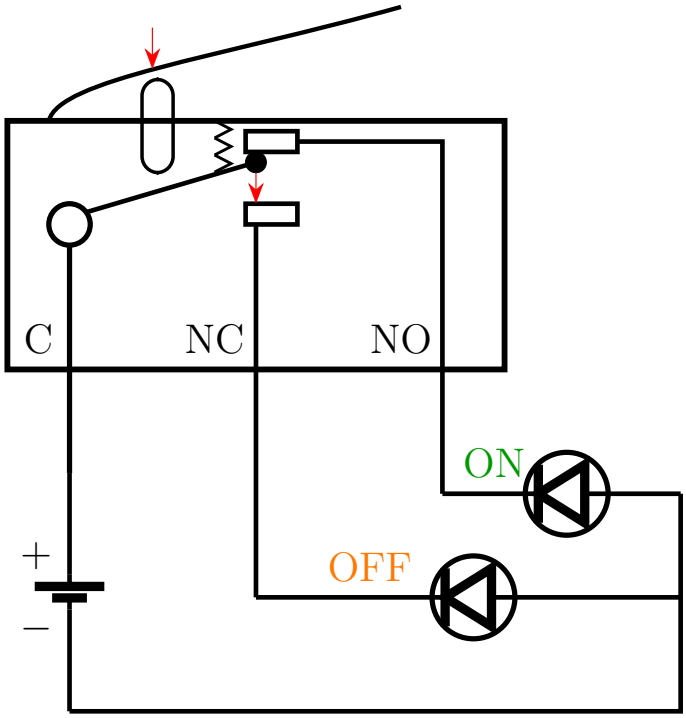

Abbildung 1.2.: Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO

Haupteinsatzgebiete:

- Endstopp-Erkennung: Schutz der Maschine vor dem Überfahren der mechanischen Grenzen.
- Referenzschalter (Homing): Der Schalter dient als „Referenzschalter“, um den Nullpunkt einer Achse (z. B. beim 3D-Drucker) zu kalibrieren [Bas].

1.5. Analyse: Vor- und Nachteile

Vorteile:

- Wirtschaftlichkeit: Sehr kostengünstig (ca. 4-5 € pro Stück) [3DJake, Bastelgarage].
- Einfachheit: Direkte Erzeugung eines binären Signals ohne aufwändige Auswerteelektronik.
- Robustheit gegen EMV: Unempfindlich gegenüber elektromagnetischen Störungen, da kein Oszillator wie bei induktiven Sensoren benötigt wird.

Nachteile:

- Mechanischer Verschleiß: Durch den physischen Kontakt unterliegen die Bauteile einer Abnutzung.
- Geschwindigkeit: Mechanische Kontakte können „prellen“ und sind langsamer als elektronische Lösungen [Her+, S. 444].
- Technologische Verbreitung: Obwohl immer noch sehr viele mechanische Endschalter eingesetzt werden, ist der berührungslose Endschalter (Sensorschalter) in nahezu allen Bereichen als Standard-Bauelement zu finden [Her+, S. 435].

1.6. Ausblick: Intelligente und vernetzte Sensorik

Die Zukunft der Positionserfassung liegt in der bidirektionalen Kommunikation. Ein moderner Standard ist hierbei IO-Link, welcher eine „feldbusunabhängige Kommunikation zwischen Sensoren und Aktoren und der Maschinensteuerung“ beschreibt [Her+, S. 510].
Während klassische Endschalter nur „1 Bit“ an Informationen liefern (Ein/Aus), können intelligente Systeme über IO-Link zusätzlich Servicedaten übertragen, „die zur Einstellung oder Diagnose eines Devices führen“ [Her+, S. 512].
Dies ermöglicht eine „vorbeugende Instandhaltung“, indem beispielsweise Verschleiß gemeldet wird, bevor es zum Maschinenausfall kommt [Her+, S. 471, 512].

1.7. Library

Es werden keine zusätzlichen Bibliotheken zur Initialisierung des Sensors benötigt.

1.8. Beispiel

Dieses Beispiel zeigt, wie der Endschalter über einen digitalen Eingang ausgelesen werden kann. Dabei wird der interne Pull-Up-Widerstand verwendet.

1.8.1. Erklärung des Beispiel-Codes

Nach der Initialisierung durch die Funktion `setup()` wird das Programm durch die Funktion `loop()` kontinuierlich ausgeführt. Diese Funktion stellt eine Endlosschleife dar, in der der Zustand des Endschalters bei jedem Durchlauf überprüft wird.[Ardg; Arde]
Der verwendete Pin wird mit der Funktion `pinMode()` als Eingang oder Ausgang konfiguriert. [Ardf] Dabei wird der Modus `INPUT PULLUP` verwendet, um den internen Pull-Up-Widerstand zu aktivieren.[Arda] Der interne Pull-Up-Widerstand sorgt dafür, dass am Eingang im Ruhezustand ein definiertes HIGH-Signal anliegt. Innerhalb der `loop()`-Funktion wird der Zustand des Endschalters mit der Funktion `digitalRead()` ausgelesen. Der gelesene Wert wird in einer Variablen gespeichert und über die serielle Schnittstelle ausgegeben.[Ardd] Die Funktionen `Serial.print()` und `Serial.println()` werden verwendet, um den aktuellen Zustand des Eingangs im Serial Monitor darzustellen.[Ardb] Durch die Verwendung des internen Pull-Up-Widerstands ergibt sich eine invertierte Logik. Im nicht betätigten Zustand des Endschalter liegt ein HIGH-Signal am Eingang an. Wird der Endschalter betätigt liegt ein LOW-Signal an. Zur besseren Lesbarkeit der Ausgabe wird eine Verzögerung mit der Funktion `delay()` erzeugt.[Ardc]

Listing 1.1.: Einfaches Beispiel zum Lesen eines mechanischen Endschalters

```
/**
 * @file MicroSwitch_BasicRead.ino
 * @brief Basic example for reading a micro switch.
*/
const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
    switch. */

/**
 * @brief Initializes the serial interface and the input pin.
 */
void setup()
{
  Serial.begin(9600);
  pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
```

```
}

/**
 * @brief Reads the switch state and prints it to the serial monitor.
 */
void loop()
{
  int switchState = digitalRead(MICRO_SWITCH_PIN);

  Serial.print("Micro switch state: ");
  Serial.println(switchState);

  delay(200);
}
```

../../Code/Beispielcode/MicroSwitch_BasicRead/MicroSwitch_BasicRead.ino

1.9. Einfacher Test-Code

Die Funktion `loop()` läuft dauerhaft und liest den Zustand des Mikroschalters ein. Da `INPUT PULLUP` verwendet wird, bedeutet LOW = gedrückt. Ist der Schalter gedrückt, wird die LED eingeschaltet, sonst ausgeschaltet.

Listing 1.2.: An- und Ausschalten einer LED mittels Endschalter

```
/**
 * @file MicroSwitch_LEDExample.ino
 * @brief Simple application example using a micro switch and an LED.

 */

const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
    switch. */
const int LED_PIN = 13;         /**< Digital output pin for the built-in
     LED. */

/**
 * @brief Initializes the input and output pins.
 */
void setup()
{
  pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
}

/**
 * @brief Switches the LED depending on the micro switch state.
 */
void loop()
{
  int switchState = digitalRead(MICRO_SWITCH_PIN);

  if (switchState == LOW)
  {
    digitalWrite(LED_PIN, HIGH);
  }
  else
  {
    digitalWrite(LED_PIN, LOW);
  }
}
```

../../Code/Beispielcode/MicroSwitch_LEDExample/MicroSwitch_LEDExample.ino

1.10. Einfache Anwendung

Das Programm führt eine Referenzfahrt mit einem Schrittmotor aus.
Im `setup()` werden die Pins für den Treiber und den Endschalter eingerichtet. Danach wird der Motortreiber aktiviert und die Referenzfahrt gestartet.
Die Funktion `oneStep()` erzeugt einen einzelnen Schritt des Motors.
Mit `moveSteps()` kann der Motor eine festgelegte Anzahl an Schritten in eine bestimmte Richtung fahren.
In `homeAxis()` fährt der Motor so lange, bis der Endschalter gedrückt wird. Danach wartet das Programm kurz und fährt den Motor einige Schritte zurück.

Listing 1.3.: Referenzfahrt einer Achse mit Endschalter

```
/**
 * @file referenzfahrt.ino
 * @brief Homing routine using a stepper motor, A4988 driver, and
    endstop.
 */

/** @brief STEP pin of the driver */
const int stepPin = 2;

/** @brief Direction pin of the driver */
const int dirPin = 5;

/** @brief Enable pin of the driver */
const int enPin = 8;

/** @brief Endstop input pin */
const int endstopPin = 9;

/** @brief Step delay in microseconds */
const int stepDelayUs = 1000;

/** @brief Number of steps for the backoff movement */
const int backoffSteps = 50;

/** @brief Initializes the hardware and starts the homing routine */
void setup() {
  pinMode(stepPin, OUTPUT);
  pinMode(dirPin, OUTPUT);
  pinMode(enPin, OUTPUT);
  pinMode(endstopPin, INPUT_PULLUP);

  Serial.begin(115200);

  digitalWrite(enPin, LOW);
  digitalWrite(dirPin, LOW);

  homeAxis();
}

/** @brief Main loop (unused in this example) */
void loop() {
}

/** @brief Executes a single motor step */
void oneStep() {
  digitalWrite(stepPin, HIGH);
  delayMicroseconds(stepDelayUs);
  digitalWrite(stepPin, LOW);
  delayMicroseconds(stepDelayUs);
}

/**
 * @brief Moves the motor by a defined number of steps
 * @param steps Number of steps to execute
```

```
 * @param dir Direction of movement (LOW/HIGH)
 */
void moveSteps(int steps, bool dir) {
  digitalWrite(dirPin, dir);
  for (int i = 0; i < steps; i++) {
    oneStep();
  }
}

/** @brief Performs the homing sequence and backoff movement */
void homeAxis() {
  while (digitalRead(endstopPin) == HIGH) {
    oneStep();
  }

  delay(200);
  moveSteps(backoffSteps, HIGH);
}
```

../../Code/Beispielcode/referenzfahrt/referenzfahrt.ino

Teil III.

Domain Knowledge - Tools

Teil IV.

Developmenmt

Teil V.

Monitoring

Teil VI.

Verification - Evaluation - Conclusion

Teil VII.

Anhang

Literaturverzeichnis

[3dj] *Creality Endschalter*. 2026. URL: https://www.3djake.de/creality-3d-drucker-ersatzteile/endschalter (besucht am 26. 04. 2026).

[Arda] *Digital Pins - Arduino Reference*. 2024. URL: https://docs.arduino.cc/language-reference/de/variablen/constants/inputOutputPullup/ (besucht am 25. 04. 2026).

[Ardb] *Serial.print() - Arduino Reference*. 2024. URL: https://www.arduino.cc/reference/de/language/functions/communication/serial/print/ (besucht am 25. 04. 2026).

[Ardc] *delay() - Arduino Reference*. 2024. URL: https://www.arduino.cc/reference/de/language/functions/time/delay/ (besucht am 25. 04. 2026).

[Ardd] *digitalRead() - Arduino Reference*. 2024. URL: https://www.arduino.cc/reference/de/language/functions/digital-io/digitalread/ (besucht am 25. 04. 2026).

[Arde] *loop() - Arduino Reference*. URL: https://www.arduino.cc/reference/de/language/structure/sketch/loop/ (besucht am 25. 04. 2026).

[Ardf] *pinMode() - Arduino Reference*. 2024. URL: https://www.arduino.cc/reference/de/language/functions/digital-io/pinMode/ (besucht am 25. 04. 2026).

[Ardg] *setup() - Arduino Reference*. URL: https://www.arduino.cc/reference/de/language/structure/sketch/setup/ (besucht am 25. 04. 2026).

[Bas] *Creality Endschalter*. 2026. URL: https://www.bastelgarage.ch/creality-endschalter-limit-switch (besucht am 26. 04. 2026).

[Her+] E. Hering u. a. *Elektrotechnik und Elektronik für Maschinenbauer*. Bd. 2. Springer. ISBN: 978-3-642-12881-3.